

The Ellipsoid Method with Quantum Subroutines

by

Benjamin R Hanzsek-Brill

B.Sc., University of Minnesota-Duluth, 2023

B.A., University of Minnesota-Duluth, 2023

Thesis Submitted in Partial Fulfillment of the
Requirements for the Degree of
Master of Science

in the
Department of Mathematics
Faculty of Science

© Benjamin R Hanzsek-Brill 2025
SIMON FRASER UNIVERSITY
Summer 2025

Copyright in this work is held by the author. Please ensure that any reproduction or re-use is done in accordance with the relevant national copyright legislation.

Declaration of Committee

Name:	Benjamin R Hanzsek-Brill
Degree:	Master of Science
Thesis title:	The Ellipsoid Method with Quantum Subroutines
Committee:	Tamon Stephen Supervisor Professor, Mathematics Ben Adcock Committee Member Professor, Mathematics

Abstract

This project begins the process of adapting the ellipsoid method of optimization to use quantum subroutines. It uses Nannicini's Sign Detection Algorithm on quantum states as well as Grover's Search Algorithm to provide substantial speedup in finding a constraint violated by a potential solution for a given LP. In specific, given an LP with an $M \times N$ constraint matrix and maximum bit length E of a single variable, the complexity of finding a violated constraint drops from $\mathcal{O}(MNE)$ classically to $\tilde{\mathcal{O}}(\sqrt{M}(\log_2(M) + N)E)$ with the quantum subroutine. This is accompanied by computer simulations of the quantum circuit to solve small linear programs.

Keywords: Linear Programming; Ellipsoid Method; Quantum Computing; Quantum Search

Acknowledgements

I have deep gratitude to Dr. Tamon Stephen for his patience and advice while acting as my advisor. This work would never have been completed without him. Immense thanks to Dr. Ben Adcock for being the reader for this project. Thank you to the faculty of and my peers at Simon Fraser University. My time with all of you has been wonderful.

Contents

Bibliography	i
Declaration of Committee	ii
Abstract	iii
Acknowledgements	iv
Table of Contents	v
1 Introduction	1
2 Background	2
2.1 Historical Notes	2
2.1.1 Linear Programming	2
2.1.2 Quantum Computing	2
2.1.3 Current State of Quantum LP	3
2.2 Ellipsoid Method	4
2.2.1 Ensuring Full Dimensionality	7
2.2.2 Numerical Analysis for Convergence	8
2.2.3 Complexity Analysis	10
2.3 Basics of Quantum Computing	10
2.3.1 A Model of Classical Computation	11
2.3.2 Quantum Computation	12
2.3.3 Circuit Model of Computation	12
2.3.4 Complexity in Quantum Computing	13
3 Quantum Algorithms	14
3.1 Interference of Quantum States	14
3.2 Grover's Search Algorithm	15
3.3 Quantum Amplitude Estimation	15
3.4 Sign Estimation	16
4 Quantum-Search for Violated Constraints	17
4.1 Interference Subcircuit	17
4.2 No False Positives	18
4.3 Complexity	19
5 Quantum-Search Ellipsoid Method	20

5.1	The Algorithm	20
5.2	Implementation	22
5.2.1	Code Implementation	22
5.2.2	Computational Results	25
5.2.3	Practicality	25
6	Conclusion	27
6.1	Results	27
6.2	Future Research	27
Appendix A Grover's Algorithm		31
Appendix B Amplitude Estimation		33
Appendix C Sign Estimation		35

Chapter 1

Introduction

We have seen in recent years the physical realization of quantum computation [CGK98] [Joh+11]. This has led to increased interest in the development of quantum algorithms. One approach to developing such algorithms has been to take existing classical algorithms and adapt them using well-known quantum techniques. This process of quantization has yielded a plethora of interesting results.

In the field of linear programming, both the simplex method [Nan22] and interior point methods [CM20] have been effectively quantized. However, the ellipsoid method, the third major method for solving LPs, has yet to receive such treatment. This is likely due to its limited practical application in industrial settings. The ellipsoid method is generally viewed as an interesting theoretical tool with historical significance. Therefore, it seems proper that this technique gets a quantization in step with its sister linear programming methods.

The goal of the work presented here was originally to derive a complete quantization of the ellipsoid method. Such an algorithm should be able to identify a given linear program as either feasible or infeasible with computational speed-up compared to the classical version. The results of this work reveal a useful quantum circuit which identifies a polytope defining constraint a certain point c might be violating. That is, given an $M \times N$ matrix A , a vector b of dimension N , and a vector c of dimension N , this quantum subroutine can find an index for which $a_i^T c < b_i$ where a_i is the i th row of A . This is one of the fundamental subroutines of the ellipsoid method.

Section 2 provides a brief overview of the historical and mathematical background to this problem including an introduction to both ellipsoid method and quantum computation. Following this, the subroutines used in the quantum algorithm will be introduced in Section 3. The final subroutine will then be described and its theoretical complexity analyzed in Section 4. Section 5 describes the process of adapting a generic LP into a quantum solvable system. The penultimate section, that is Section 5.2, will be an implementation of this algorithm using the Qiskit simulator and empirical analysis of its behavior compared to its classical counterpart. The final section will be a discussion on the successes and failures of this algorithm both theoretically and practically as well as discussion of possible future research.

Chapter 2

Background

2.1 Historical Notes

2.1.1 Linear Programming

Linear Programs (LPs) were first developed by Soviet mathematician Leonid Kantorovich as a way to generate more efficient factory schedules [Kan60]. The model rapidly gained prominence on the mathematical stage in WWII as an efficient tool for military logistics, resource use, and transport scheduling [SZ15].

George Danzig invented the Simplex Method (SM) in 1947 while working for the U.S. Air Force [Dan82]. In the later 20th century, the Simplex method found use in industrial applications. With the adoption of digital computers, LP solvers became some of the most important programs available. As LP algorithms became more efficient, LPs became more widely adopted, leading to demand for even faster algorithms. In 1979, Leonid Khachiyan proved that LPs could be solved in polynomial time using the Ellipsoid Method (EM) [Kha80]. However, Simplex generally out performs EM in practice. Narendra Karmarkar, using some of Khachiyan's techniques, invented Interior Point Methods (IPMs) to solve LP in 1984 [Str87].

While EM and IPMs are closely related, they are distinct in several ways. One can see this geometrically by the way the two methods initialize. The EM first generates an ellipse which contains parts of the feasible region of an LP and then works inward to a feasible point; IPMs begin inside a feasible region and work outward to an optimal point. Generally, IPMs are competitive with SM and can out perform it on sparse problem.

Linear programs are used widely in both academic and industrial settings to this day.

2.1.2 Quantum Computing

There has been a paradigm shift in computer science as quantum computing has become physically achievable on a small scale. Quantum Computing became a field of study in the late 1970s and early '80s when physicists, either side of the Iron Curtain, suggested using new quantum tools in computation. Yuri Manin [Man80] and Richard Feynman [Fey82] were among the first to propose quantum computers as an alternative to classical computation. In 1982, Paul Benioff's paper on the quantum Turing machine formalizes quantum computing as a field of study [Ben82].

Over the next two decades, important theoretical advances were made in the field of quantum computer science. Grover’s search algorithm shows the potential for quadratic speed-up in unstructured search problems [Gro96]. Peter Shor’s factoring algorithm provides exponential speed-up while factoring large numbers [Sho97]. If realized, this algorithm would render standard encryption routines, such as RSA, ineffective.

In 1998, the first physical realization of a quantum computer was used to perform Grover’s search over a space of two qubits [CGK98]. In 2011, the company D-Wave Systems announced the first commercially available quantum annealing device with 128 qubits [Joh+11]. A quantum annealer is a device which is able to solve combinatorial optimization problems but cannot implement all quantum algorithms. In 2025, both Microsoft [Aas+25] and IBM [Yod+25] announced road maps to fault tolerant and scalable quantum systems within the decade, though demonstrations of these claims are yet to be made public.

2.1.3 Current State of Quantum LP

The use of quantum computing in linear programming has emerged as a new and exciting field within the last decade. As stated in the introduction, both the simplex method and interior point methods have had quantum adaptations.

The simplex method was originally adapted in 2019 by Giacomo Nannicini. This successfully runs an iteration of the simplex method in quadratically faster than the classical case [Nan22]. This was done with a combination of the Harrow–Hassidim–Lloyd (HHL) quantum linear system algorithm (QLSA) and Grover’s search. In 2021, a quantum-search algorithm based on Grover’s was simulated with IBM’s Quantum Experience in order to find entering and exiting variables for the basis [JAJ21]. In 2023, benchmarking via classical simulation was performed on Nannicini’s algorithm which demonstrated it required quantum hardware beyond the current state of the technology [Amm+23].

Nannicini developed the NFP subroutine which detects the sign of a quantum amplitude, described in detail in Section 3.4, in order to determine the reduced cost of a column entering the basis for the simplex method. This same subroutine is used in this work in order to find a row constraint, if any, which is violated by the potential solution vector which is the center of the ellipse.

The interior point method was first quantized by Kerenidis and Prakash resulting in a linear speed up of the IPM [KP18]. This original method was refined in 2021 to guarantee the convergence of the system as the original implementation did not account for the inexact calculations performed by quantum computers [Aug+23]. This is accomplished by solving the Newtonian Linear System necessary for the central-path following part of IMPs with the HHL QSLA each step of the method. It also uses quantum tomography to reconstruct the information needed to solve both the primal and dual problems of the LP.

The ellipsoid method has yet to be quantized likely because of its limited practical applications. However, the structure of the ellipsoid method is well suited to a quantum adaptation. This method has three major subroutines: search, center projection, and ellipse projection. It is well known that quantum computer provide speed-up in search problems and the work here shows that such speed-up is possible for the ellipsoid method.

2.2 Ellipsoid Method

First, we must define the problem we are trying to solve. Given an $M \times N$ matrix A and a vector of length M named b , called the *constraint matrix* and *constraint vector* respectively, we want to determine if there exists a vector x such that $Ax \geq b$. We define the polytope $P := \{y \in \mathbb{R}^N : Ay \geq b\}$ to be the *feasible region*.

Definition 1. A linear program defined by A and b is **feasible** if the polytope $P := \{y \in \mathbb{R}^N : Ay \geq b\}$ is non-empty. Otherwise, the LP is **infeasible**.

The strategies used to solve LPs are called “methods” rather than algorithms since they involve black box subroutines which can be solved any number of ways. The classical ellipsoid method requires linear algebra which can be run on classical computers but can be more easily solved with quantum systems.

Definition 2. An **ellipsoid** in $\mathbb{R}^N$ is defined by a center point c and non-negative definite matrix D and is the region $E(c, D) := \{y \in \mathbb{R}^N : (y - c)^T D^{-1} (y - c) \leq 1\}$.

Theorem 1. The **ellipsoid method for feasibility** is able to determine if a given LP is feasible or not in polynomial time complexity.

Let there be an LP defined by A, b with polytope P . The ellipsoid method establishes a ball $B_0 = E_0 = E(c_0, D_0)$ centered on the origin such that if P is non-empty, then $E_0 \cap P$ is non-empty. In fact, if P is non-empty, then B_0 will contain all the extreme points of P .

The EM then checks if the center of the ball c_0 is a point of P . If so, the method outputs $x = c_0$. Else, the method finds a row a_i of the matrix A such that $a_i^T c_0 < b_i$. This row a_i is called a *violated constraint*. The method cuts the ball in half on a hyperplane which is parallel to the constraint a_i and passes through c_0 . It then generates a new ellipsoid $E_1 = E(c_1, D_1)$ which is an ellipsoid containing the intersection of the previously mentioned half-space and E_0 . It does the same for each iteration t and ellipsoids E_t & E_{t+1} .

The volume of E_{t+1} is guaranteed to be less than the volume of E_t . The ellipsoid method repeats this process until the stopping criterion are met. The first stopping condition is the maximum number of iterations is reached. The other is that the method finds some point $x_t \in P$ which is feasible. The illustration in figure 2.1 shows the ellipsoid method determining that the LP defined by

$$A = \begin{pmatrix} 1 & 0 \\ -1 & 0 \\ 0 & 1 \\ 0 & -1 \end{pmatrix}, b = \begin{pmatrix} 1 \\ -2 \\ 1 \\ -2 \end{pmatrix}$$

is feasible in three steps.

The ellipsoid method initializes with a few important values. Define $U = \mathbf{max}\{A, b\}$ and let $u = \lceil \log_2(U) \rceil$ to be the number of bits needed to express each entry of A and b . The smallest non-zero volume P can take is called v . The volume of the ball which contains all the extreme points of P when centered on the origin is called V . Both v and V are dependent upon U and N . The value r is the radius of the ball of volume V . The maximum number of iterations is called t_{max} . In the algorithmic description, the sign $\approx$ represents rounding the calculation to a determined precision p .

As part of this project, I have supplied python code which executes the classical ellipsoid method for feasibility [Han25]. For a more in-depth treatment of the ellipsoid method and

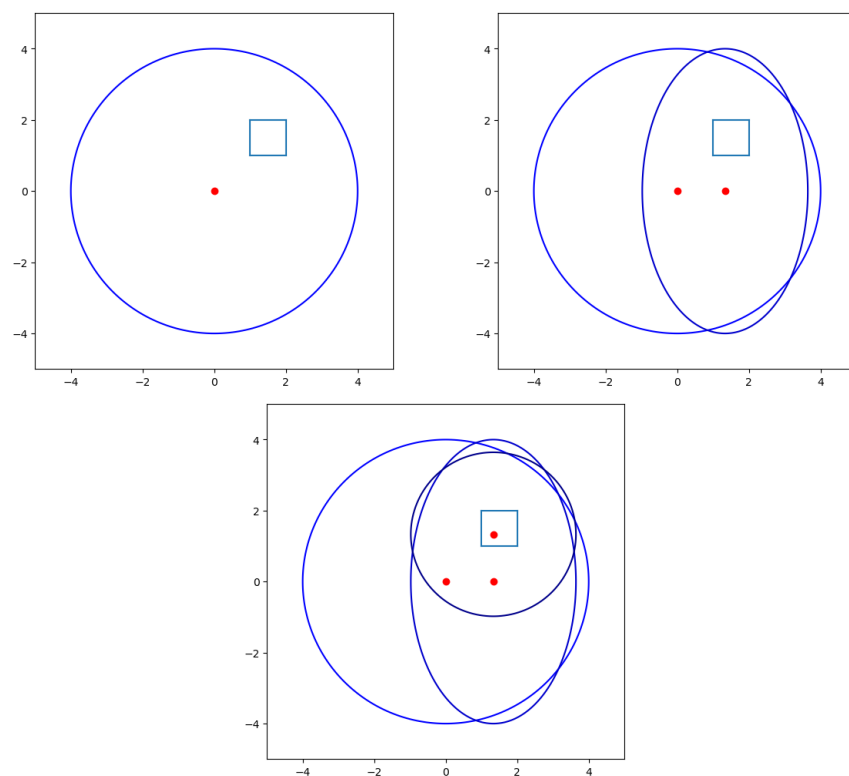


Figure 2.1: The ellipsoid method in two-dimensions.

theoretical justification the the initialization values, read *Introduction to Linear Optimization* [BT97, Ch. 8] or *Geometric Algorithms and Combinatorial Optimization* [GLS88, Ch. 3].

Algorithm 1 Classical Ellipsoid Method for Feasibility

```

1: Input: An  $M \times N$  matrix  $A$  and an  $M \times 1$  vector  $b$  which define  $P := \{x \in \mathbb{R}^N : Ax \geq b\}$ .
2: Output: A vector  $x$  of length  $N$  such that  $Ax \geq b$  or a statement that  $\text{vol}(P) = 0$ .

3: procedure INITIALIZE
4:    $v := N^{-N}(NU)^{-N^2(N+1)}$ 
5:    $V := (2N)^N(NU)^{N^2}$ 
6:    $U := \max(A, b)$ 
7:    $u := \lceil \log_2(U) \rceil$ 
8:    $r := \sqrt{N} \cdot 2^{(N^2+N)u-N^2}$ 
9:    $t_{\max} := \lceil 5N|\log_2(v)| + 5N^2|\log_2(2r)| \rceil$ 
10:   $c_0 := \emptyset$ 
11:   $D_0 := r^2 I$ 
12:   $E_0 := (c_0, D_0)$ 
13:   $t := 0$ 

14: procedure MAIN LOOP
15:   if  $t \geq t_{\max}$ , then
16:     Output:  $\text{vol}(P) = 0$ 
17:   if  $\forall i \in \mathbb{N}_M; a_i^T c_t \geq b_i$ , then
18:     Output:  $x := c_t$ 
19:   if  $\exists i$  s.t.  $a_i^T c_t < b_i$ , then
20:      $c_{t+1} := c_t + \frac{1}{N+1} \cdot \frac{D_t a_i}{\sqrt{a_i^T D_t a_i}}$ 
21:      $D_{t+1} := \frac{2N^2+3}{2N^2} (D_t - \frac{2}{N+1} \cdot \frac{D_t a_i a_i^T D_t}{a_i^T D_t a_i})$ 
22:      $t := t + 1$ 

```

2.2.1 Ensuring Full Dimensionality

It may be the case that the feasible region is not full dimensional in $\mathbb{R}^N$. This means the volume of the feasible region is 0 and the ellipsoid method would be unable to distinguish such as either feasible or infeasible. If this occurs, it is possible to slightly alter the constraint vector b to make the problem solvable with ellipsoid method.

In particular, let $\delta = \frac{1}{2(N+1)}((N+1)U)^{-(N+1)}$ and define a new polytope $P' = \{x \in \mathbb{R}^N : Ax \geq b - \delta \mathbf{1}\}$ where $\mathbf{1}$ is the vector of all 1s. The space P' is non-empty if and only if $P = \{x \in \mathbb{R}^N : Ax \geq b\}$ is non-empty. The following proof is adapted from Bertsimas and Tsitsiklas [BT97, §8.3].

Suppose P is empty. Create the linear program,

$$\begin{aligned} \text{MIN: } & \mathbf{0}^T x \\ \text{ST: } & Ax \geq b \end{aligned}$$

which is infeasible. Therefore, its dual

$$\begin{aligned} \text{MAX: } & b^T y \\ \text{ST: } & A^T y = \mathbf{0} \\ & y \geq 0 \end{aligned}$$

will have an unbounded optimum. Therefore, there exists some basic feasible solution y' such that $A^T y' = \mathbf{0}$ and $b^T y' = 1$. We also know that $\forall i, y'_i \leq ((N+1)U)^{N+1}$ from the previous section. Since y' is basic feasible, we know $\sum_{i=1}^M y'_i \leq (N+1)((N+1)U)^{N+1}$ since at most $N+1$ entries are non-zero. Thus,

$$\begin{aligned} (b - \delta \mathbf{1})^T y' &= 1 - \delta \sum_{i=1}^M y'_i \\ &\geq 1 - \delta[(N+1)((N+1)U)^{N+1}] \\ &= 1 - \left[\frac{1}{2(N+1)}((N+1)U)^{-(N+1)}\right][(N+1)((N+1)U)^{N+1}] \\ &= \frac{1}{2} \end{aligned}$$

This means the objective function of the dual does not go to zero when we perturb b by δ . Since the constraints of A are unchanged, this means the dual problem still has an unbounded objective. Therefore, the primal problem with constraint vector $b - \delta \mathbf{1}$ is still infeasible.

Suppose instead that there exists x' such that $Ax' \geq b$ meaning P is non-empty. Let p be a vector such that $|p_j - x'_j| \leq \delta(NU)^{-1}$ for all j . Therefore for all integers $1 \leq i \leq M$,

$$\begin{aligned} \sum_{j=1}^N a_{ij}p_j &\geq \sum_{j=1}^N a_{ij}x'_j - \frac{\delta}{NU} \sum_{j=1}^N |a_{ij}| \\ &= b_i - \frac{\delta}{NU} NU \\ &= b_i - \delta \end{aligned}$$

Therefore, p belongs to the perturbed polytope P' . Since the set $\{p \in \mathbb{R}^N : |p_j - x'_j| \leq \delta(NU)^{-1}\}$ forms a fully dimensional cube in $\mathbb{R}^N$, this means P' will also be fully dimensional and contain that cube.

2.2.2 Numerical Analysis for Convergence

We need $E = 8\lceil 5N|\ln(v)| + 5N^2|\ln(2r)| \rceil \sim \mathcal{O}(N^3\ln(U))$ binary digits in order for the ellipsoid method to converge. That is, this numerical precision is needed in order to guarantee the volume of the ellipsoid shrinks every iteration. This is important to consider since the calculation of a each new ellipse requires taking the square root of a rational number which could require infinite storage space. It must be shown that finite representations of the number lead to finite running times.

The stated value E ensures that $\mathbf{Vol}(E_{t+1}) \leq e^{-1/5N} \mathbf{Vol}(E_t)$. Proof of this is taken from Grötschel, Lovász, and Schrijver [GLS88, §3.2].

Recall, that the volume of an ellipse E_t defined by matrix D_t is $\sqrt{\det(D_t)}\mathbf{vol}(B_N)$ where B_N is the unit ball in $\mathbb{R}^N$. Additionally, we define

$$D_{t+1} := \frac{2N^2 + 3}{2N^2} \left(D_t - \frac{2}{N+1} \cdot \frac{D_t a_i a_i^T D_t}{a_i^T D_t a_i} \right)$$

where $\approx$ indicates rounding each entry to E binary digits. We use D'_{t+1} to indicate the unrounded matrix. Let $D_t = Q^2$ which is possible since all D_t are non-negative definite.

Each iteration the volume of the ellipse shrinks by a factor,

$$\frac{\mathbf{vol}(E_{t+1})}{\mathbf{vol}(E_t)} = \sqrt{\frac{\det(D_{t+1})}{\det(D_t)}} = \sqrt{\frac{\det(D'_{t+1})}{\det(D_t)}} \sqrt{\frac{\det(D_{t+1})}{\det(D'_{t+1})}}$$

We observe,

$$\begin{aligned}
\frac{\det(D'_{t+1})}{\det(D_t)} &= \det(D'_{t+1}Q^{-1}Q^{-1}) \\
&= \det(Q^{-1}D'_{t+1}Q^{-1}) \\
&= \det(Q^{-1}[\frac{2N^2+3}{2N^2}(D_t - \frac{2}{N+1} \cdot \frac{D_t a_i a_i^T D_t}{a_i^T D_t a_i}])Q^{-1}) \\
&= \det(Q^{-1}[\frac{2N^2+3}{2N^2}(QQ - \frac{2}{N+1} \cdot \frac{QQ a_i a_i^T QQ}{a_i^T QQ a_i}])Q^{-1}) \\
&= \det(\frac{2N^2+3}{2N^2}(\mathbb{I} - \frac{2}{N+1} \cdot \frac{Q a_i a_i^T Q}{a_i^T QQ a_i})) \\
&= (\frac{2N^2+3}{2N^2})^N \det(\mathbb{I} - \frac{2}{N+1} \cdot \frac{Q a_i a_i^T Q}{a_i^T QQ a_i})
\end{aligned}$$

Because $\frac{Q a_i a_i^T Q}{a_i^T QQ a_i}$ has rank 1 and trace 1, the matrix $I - \frac{Q a_i a_i^T Q}{a_i^T QQ a_i}$ has eigenvalues $\{1, \dots, 1, 1 - \frac{2}{N+1}\}$. Therefore,

$$\begin{aligned}
\frac{\det(D'_{t+1})}{\det(D_t)} &= (\frac{2N^2+3}{2N^2})^N \frac{N-1}{N+1} \\
&\leq e^{\frac{3}{2N}} e^{\frac{-2}{N}}; \text{ since } 1+x \leq e^x \text{ and } (1 + \frac{2}{N-1})^N > e^2 \\
&= e^{\frac{-1}{2N}}
\end{aligned}$$

For the next step of the proof, we establish some facts. We say that for a matrix B , $\|B\|$ indicates the spectral norm of B . It is a known fact that for $N \times N$ matrix B , $\det(B) \leq \|B\|^N$. It can be shown that $\|(D'_{t-1})^{-1}\| \leq 3r^{-2}4^t$ where $r = \sqrt{N} \cdot 2^{(N^2+N)u-N^2}$ [GLS88, §3.2]. Therefore,

$$\begin{aligned}
\frac{\det(D_{t+1})}{\det(D'_{t+1})} &= \det(D_{t+1}(D'_{t+1})^{-1}) \\
&= \det(I + D_{t+1}(D'_{t+1})^{-1} - I) \\
&= \det(I + (D'_{t+1})^{-1}(D_{t+1} - D'_{t+1})) \\
&\leq \|I + (D'_{t+1})^{-1}(D_{t+1} - D'_{t+1})\|^N \\
&\leq (\|I\| + \|(D'_{t+1})^{-1}\| \cdot \|D_{t+1} - D'_{t+1}\|)^N \\
&\leq (1 + (\frac{1}{3}r^{-2})(N2^{-\delta}))^N \\
&\leq (1 + (r^{-2})(N2^{-\delta}))^N \\
&\leq \exp\{N^2 r^{-2} 2^{2N-\delta}\} \\
&\leq e^{\frac{1}{10N}}
\end{aligned}$$

This means,

$$\frac{\text{vol}(E_{t+1})}{\text{vol}(E_t)} \leq \sqrt{e^{\frac{-1}{2N}}} \sqrt{e^{\frac{1}{10N}}} = e^{\frac{-1}{5N}}.$$

Thus, the ellipsoid is guaranteed to shrink every iteration and will terminate.

2.2.3 Complexity Analysis

For the purposes of this work, we will not analyze the initialization procedure since it is the same in both the classical and quantum algorithms. It is also negligible in its effect on the complexity of the system.

To check if a row constraint is violated requires $a_i^T c$ to be calculated. This takes N operations each with possible complexity of $\mathcal{O}(E)$. The value E is the maximum number of bits required to encode each entry of c and intermediate matrices D_t . There are M rows resulting in a total complexity of $\mathcal{O}(MNE)$.

To calculate a new ellipse requires $\mathcal{O}(N^2E)$ operations, since $D_t a_i$, $a_i^T D_t a_i$, and $D_t a_i a_i^T D_t$ are dominated by vector-matrix transformations. Each operation has complexity on the scale of $\mathcal{O}(E)$. This results in a total complexity of $\mathcal{O}((MN + N^2)E)$ for a single step of the classical ellipsoid method.

It is guaranteed that over a single iteration, the volume of ellipsoid will decrease. Specifically, we know $\text{vol}(E_{t+1}) \leq e^{-1/5N} \text{vol}(E_t)$ [GLS88, §3.2]. Since we have a starting volume $V = (2N)^N (NU)^{N^2}$ and a smallest possible volume $v = N^{-N} (NU)^{-N^2(N+1)}$, we must find k such that $\exp\{\frac{-k}{5N}\} V \leq v$ in order to determine the number of iterations that the ellipsoid method must run. One will find that $k = (5N)[N \ln(2) + N \ln(N) + (N^3 + 2N^2) \ln(NU)]$ is the solution. This means it takes $\mathcal{O}(N^4 \ln(NU))$ iterations to run the ellipsoid method.

If one perturbs the polyhedron to ensure full dimensionality, then the complexity of the method goes up to $\mathcal{O}(N^6 \ln(NU))$ [BT97, §8.3]. This is because when we perturb the polytope by δ , we increase both the largest possible volume and decrease the smallest possible volume of the polytope we are searching for by a factor of N . Thus, we need an extra factor of N^2 in order to ensure convergence.

Therefore, the total complexity of the ellipsoid method is

$$\mathcal{O}((MN + N^2)E \cdot N^6 \ln(NU)).$$

2.3 Basics of Quantum Computing

This section provides a brief description of the language used in quantum computation. For a more detailed introduction to quantum computing, read [Nan20] and read [NC00] for a complete foundation in the field.

If we want to explain how quantum computation differs from classical computation, we will need a quick foundation in the basics of theoretical computing.

Definition 3. A *computer* is a system which holds organized information in the form of a state and moves between states over distinct time steps in a predictable fashion.

There are *digital* computers, which have a finite number of states, and *analog* computers, which have a continuous range of states. If a user is able to dictate how these state evolutions take place, then the computer is *programmable*. Models of computing include *classical computation*, in which all moves are deterministic, *probabilistic computation*, in which all moves are probabilistic, or *quantum computation*, in which moves are described by quantum-state evolution.

2.3.1 A Model of Classical Computation

A classical computer works on bits which hold discrete values 0 and 1. The state of a classical computer then is a binary string. We can represent these bits as vectors using Dirac notation. The 0 state of a bit will be called $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ and the 1 state will be called $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. The reason for this representation will be made clear in the next section.

Suppose we have a classical computer with three bits in the states $|b_1\rangle = |0\rangle, |b_2\rangle = |1\rangle, |b_3\rangle = |1\rangle$. We could describe the state of the computer in a *reductionist* way as we just did by listing the state of each bit *locally*. However, it is also possible to describe the *global* system as a sum of independent parts.

We use the *tensor product* $\otimes : \mathbb{C}^m \times \mathbb{C}^n \rightarrow \mathbb{C}^{mn}$ to describe the global state of multiple classical bits. Thankfully, this is an associative function. Thus, we can describe our system as $(\begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}, \begin{pmatrix} 0 \\ 1 \end{pmatrix}) = (|b_1\rangle, |b_2\rangle, |b_3\rangle)$ or as

$$\begin{aligned} |b_1, b_2, b_3\rangle &= |b_1\rangle \otimes |b_2\rangle \otimes |b_3\rangle \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \begin{pmatrix} 0 \\ 1 \end{pmatrix} \\ 1 \begin{pmatrix} 0 \\ 1 \end{pmatrix} \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} \begin{pmatrix} 0 \\ 0 \end{pmatrix} \\ \begin{pmatrix} 0 \\ 1 \end{pmatrix} \end{pmatrix} \\ &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \otimes \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \\ &= \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}^T \\ &= |011\rangle \end{aligned}$$

Since a bit is always described by a vector of length 2, the global state is described by a vector whose length 2^n where n is the number of bits. It should also be seen that given any binary string of length n , we can encode it as a standard basis vector for the space $\mathbb{C}^{2^n}$. It is common practice to represent binary strings as their decimal representation so long as we know the dimension of our space. This is useful because that string is also represented by the m^{th} standard basis vector indexed from zero. For example, given the state '000' we could write this as $|000\rangle$ or $|0\rangle$ or $(1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)^T$. As above, we can express the state '011' as $|011\rangle = |3\rangle = (0 \ 0 \ 0 \ 1 \ 0 \ 0 \ 0 \ 0)^T$.

We are able to describe the state evolution of a classical computer as a series of matrix multiplications acting on bits represented by the vectors $|0\rangle$ and $|1\rangle$. Therefore, the language of linear algebra can be used to describe computational systems.

2.3.2 Quantum Computation

The best method of describing the state evolution of quantum systems is with the language of linear algebra. The smallest unit of quantum information is the *qubit*. A single qubit may be either $|0\rangle$ or $|1\rangle$ as with classical bits. But a qubit can store more information since it can exist in a state of *superposition*. This is represented as $|\Psi\rangle = \alpha|0\rangle + \beta|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$ such that $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$. The values α and β are called the *amplitudes* of the quantum state.

The power of quantum computing is the fact qubits can exist in the state of superposition. The drawback of quantum computing is that we are never able to directly observe superposition. Upon observation or *measurement* of a qubit, it will collapse to state $|0\rangle$ or $|1\rangle$. Thankfully, this is not a chaotic process. If we take the state $|\Psi\rangle$ from above and measure it, we will observe the state $|0\rangle$ with probability $|\alpha|^2$ and state $|1\rangle$ with probability $|\beta|^2$. If we have a system of n qubits, we can represent the system as a vector of $\mathbb{C}^{2^n}$. If the amplitude of the i th entry of such a vector is α_i , then the probability measurement returns the binary string of length n corresponding to i is $|a_i|^2$. We index from 0 meaning $m \in \mathbb{N}_{2^m-1}$.

If a state can be represented as a pure tensor product of multiple qubits, then it is said to be *unentangled*. However, if a state can't be represented this way then it is called *entangled*. This means that the measured result of some qubit in a quantum state is correlated with the result of some other qubit. This is why it is important we have the ability to describe the global state of a quantum system. If we were only able to act on qubits locally, then there would be no advantage gained as compared to classical computing.

The classic example of entanglement is the two-qubit *Bell State*:

$$|\Phi_B\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 0 & 1 \end{pmatrix}^T.$$

This state will return either $|00\rangle$ or $|11\rangle$ upon measurement. This means if we only measure one qubit, we will know the state of the other qubit.

We say a classical vector is *preparable* if it is a unit vector in $\mathbb{C}^N$ for some $N = 2^n$. If the largest entry of a preparable vector a has bit length E , then encoding a is accomplished with $\mathcal{O}(\text{poly}(\log_2(N))E) \sim \tilde{\mathcal{O}}(E)$ gates.

Note, that we represent quantum system with matrix algebra but a quantum computer doesn't actually perform any matrix operations. Passing a qubit through a *quantum logic gate*, or simply *quantum gate*, is a single operation. Any unitary matrix in $\mathbb{C}^{2^n \times 2^n}$ can be encoded as a quantum gate.

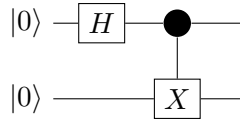
2.3.3 Circuit Model of Computation

The *circuit model* of quantum computing is accepted as standard since it provides a useful visualization of any quantum algorithm. Each qubit is represented by a wire while each operation is represented by a gate. It's important to note that wires cannot split or

merge. That is, if the input state of a circuit is n qubits long, then every state of that circuit will be n bits long.

It initializes with all qubits in the state $|0\rangle$, applies the gates of the operation in sequence, and then measures the final state of the computer. Presented below is a circuit which produces $|\Phi_B\rangle$ with only two operations. The first is the Hadamard gate H , which sends $|0\rangle$ to the state $\frac{1}{\sqrt{2}}[|0\rangle + |1\rangle]$, and the second is the $CNOT$ gate, which performs a bit-flip on the second qubit iff the first qubit will be measured in state $|1\rangle$.

$$\mathbf{CNOT} \cdot (H \otimes I) \cdot |00\rangle = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} = |\Phi_B\rangle$$



Theoretically, one could write any given quantum circuit as a single large unitary matrix which sends the input $|0^n\rangle$ directly to the desired output. Of course, any such matrix would be of little use to human readers. However, it allows for, inefficient, simulation of quantum systems on classical devices.

2.3.4 Complexity in Quantum Computing

As with any field of computation, we need some way to describe the complexity of algorithms. If a unitary gate acts on a single qubit, then we say it has *gate complexity* $\mathcal{O}(1)$. The *space complexity* of a circuit is the number of qubits needed to produce it. Both of these are directly comparable to the complexity of classical computers. If $A < B$ and a classical algorithm can solve a problem with complexity $\mathcal{O}(B)$ and a quantum algorithm can solve it with complexity $\mathcal{O}(A)$, then we can claim that the quantum computer offers an advantage over the classical one. Of course, the actual speed of quantum hardware might be much slower than that of classical hardware. This means that a quantum algorithm with a low complexity will not outperform a classical computer [Amm+23].

While analyzing complexity in the rest of this work, we make an assumption which is common within the field. We assume that encoding a classical vector of dimension $N = 2^n$ happens in $\mathcal{O}(\mathbf{poly}(n))$ time. For this reason, we often use $\tilde{\mathcal{O}}$ notation which suppresses $\mathbf{poly}(n)$ values. That is, $\mathcal{O}(\mathbf{poly}(n)Q) \sim \tilde{\mathcal{O}}(Q)$ for any value Q .

Chapter 3

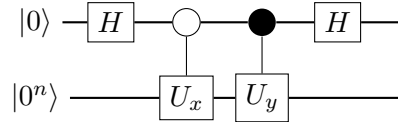
Quantum Algorithms

What follows is a collection of known quantum algorithms that we will use to build the our quantum-search subroutine for ellipsoid method.

3.1 Interference of Quantum States

Theorem 2. *Given two gate U_x and U_y able to encode states $|x\rangle$ and $|y\rangle$, there exists a method of producing a quantum state $|\psi\rangle$ such that the probability of observing $|0\rangle$ upon measuring the first qubit of $|\psi\rangle$ is $p = \frac{1}{2}(1 + \mathbf{Re}(\langle x|y\rangle))$ [Zha+19].*

Since we assume fast encoding, the cost of this process is small. In the figure below, the gates U_x and U_y encode states $|x\rangle$ and $|y\rangle$ respectively.



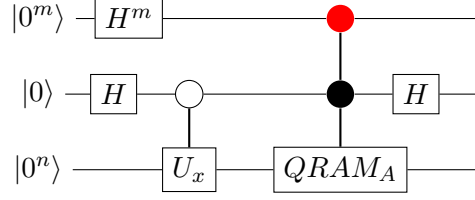
We observe,

$$\begin{aligned}
 |\psi\rangle &= (H \otimes I)(C_1 Y)(C_0 X)(H \otimes I)|0\rangle|0^n\rangle \\
 &= (H \otimes I)(C_1 Y)(C_0 X)\left[\frac{1}{\sqrt{2}}|0\rangle|0^n\rangle + \frac{1}{\sqrt{2}}|1\rangle|0^n\rangle\right] \\
 &= (H \otimes I)\left[\frac{1}{\sqrt{2}}|0\rangle|x\rangle + \frac{1}{\sqrt{2}}|1\rangle|y\rangle\right] \\
 &= \frac{1}{2}|0\rangle(|x\rangle + |y\rangle) + \frac{1}{2}|1\rangle(|x\rangle - |y\rangle)
 \end{aligned}$$

The probability that we measure $|0\rangle$ on the first qubit will be $\frac{1}{2}(1 + \mathbf{Re}(\langle x|y\rangle))$. In fact, the state of the whole system will be

$$\frac{1}{2}|0\rangle\left(\sum_{i=0}^n (x_i + y_i)|i\rangle\right) + \frac{1}{2}|1\rangle\left(\sum_{i=0}^n (x_i - y_i)|i\rangle\right)$$

If we have access to quantum random access memory (QRAM), then we are able to call and encode classical data using quantum superposition. We will let the $QRAM_A$ gate encode $|a_i\rangle$, the quantum encoding of the i th row of A , when called by the index $|i\rangle$.



Since we assume fast encoding, the main gate complexity of this circuit will be in the creating the mixed state with the H^m gates. Since $m = \log_2(M)$, where M is the number of rows of A , our gate complexity will be $\tilde{O}(\log_2(M)E)$. Here, E is the maximum number of bits needed to encode the entries of a_i and c .

3.2 Grover's Search Algorithm

Theorem 3. *Given a boolean function $f : \{0,1\}^m \rightarrow \{0,1\}$ with μ binary strings x such that $f(x) = 1$ and oracle gate U_f implementing f , there exists a quantum algorithm able to produce a state y such that $f(y) = 1$ with $\mathcal{O}(\sqrt{\frac{2^m}{\mu}})$ calls to U_f [NC00, §6.1].*

This is one of the most well-known and important algorithms in the field of quantum computing. Grover's search algorithm can perform unstructured search over a set of indices quadratically faster than classical search. That is, suppose there is a function f such that for exactly one binary string i of length m , out of all possible length m binary strings, such that $f(i) = 1$. For all $j \neq i$, $f(j) = 0$. Let $M = 2^m$.

On a classical computer, one would expect to make $\mathcal{O}(M)$ calls to the function f in order to find the index i . However, using Grover's Search, one can identify i with only $\mathcal{O}(\sqrt{M})$ calls. If instead there are q binary strings such that $f(i) = 1$, Grover's Search requires $\mathcal{O}(\sqrt{\frac{M}{q}})$ calls to f . Proof of this is in appendix A.

3.3 Quantum Amplitude Estimation

Theorem 4. *Given a quantum gate $\mathcal{A}$ such that $\mathcal{A}|0\rangle = |\psi\rangle$ and an index k , there exists a method to determine the amplitude ψ_k without direct measurement of $|\psi\rangle$ [Bra+02].*

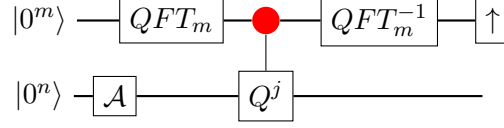
Suppose we have some boolean function $f : \{0,1\}^m \rightarrow \{0,1\}$ and some quantum state $|\psi\rangle = \mathcal{A}|0\rangle$. We say a binary string x is *good* if $f(x) = 1$. We want to estimate a , the probability that, upon measurement, $|\psi\rangle$ collapses to a good state. There exists a quantum algorithm capable of returning $\tilde{a}$, the closest approximation of a with a given number of qubits.[Bra+02].

First, the quantum Fourier transform on m qubits to be

$$QFT_m|j\rangle = \frac{1}{\sqrt{M}} \sum_{k=0}^{M-1} e^{2\pi i j k \frac{1}{M}} |k\rangle.$$

One may observe this is both a unitary transformation and corresponds to the discrete Fourier transform [NC00, §5.1].

Let B be a unitary operator on n qubits. Then the $\Lambda_m(B)$ operator is defined such that $\Lambda_m(B)|j\rangle|y\rangle = |j\rangle U^j|y\rangle$ where $|y\rangle$ is any state on n qubits and $0 \leq j < M$. If $|y\rangle$ is an eigenvector of U with eigenvalue $e^{2\pi iz}$, then $\Lambda_m(U)|j\rangle|y\rangle = e^{2\pi izj}|j\rangle|y\rangle$.



The above circuit will output the state $|\tilde{a}\rangle$ which is an m bit approximation of a such that

$$|\tilde{a} - a| \leq 2\pi \frac{\sqrt{a(1-a)}}{m} + \frac{\pi^2}{M}$$

with probability at least $\frac{8}{\pi^2}$. Proof of this statement is in appendix B.

3.4 Sign Estimation

Theorem 5. *Given a quantum gate $\mathcal{A}$ such that $\mathcal{A}|0\rangle = |\psi\rangle$ and an index k , there exists a method to determine the sign of ψ_k without direct measurement of $|\psi\rangle$ or estimation of the amplitude itself [Nan22].*

Using quantum amplitude estimation, one can predict with high probability if the amplitude of a quantum state is positive or negative to within a given ϵ . This requires a number of gates which is logarithmic in terms of ϵ [Nan22].

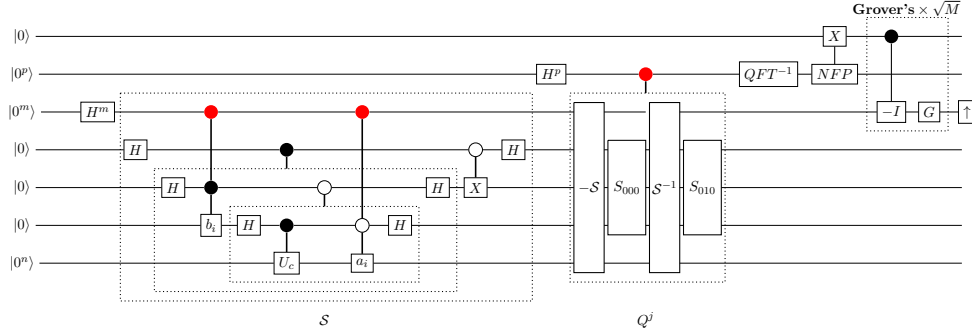
Suppose we have a unitary U_ψ such that $U_\psi|0^n\rangle = |\psi\rangle$ and unitary U_k which encodes the bit string k as quantum state $|k\rangle$. If we run the interference routine using U_ψ and U_k followed by amplitude estimation where the state $|0\rangle \otimes |k\rangle$ is considered good, then using $p = \lceil \log(\frac{\sqrt{3\pi}}{\epsilon}) + 2 \rceil$ qubits of accuracy allows us to determine the sign of ψ_k .

Suppose the amplitude estimation routine returns the bit string a . If $0.a < \frac{1}{6} - \frac{2\epsilon}{\sqrt{3\pi}}$, then $\psi_k < -\epsilon$. If $\frac{1}{6} - \frac{2\epsilon}{\sqrt{3\pi}} \leq 0.a < \frac{1}{2}$, then $\psi_k \geq -\epsilon$. If $\frac{1}{2} \leq 0.a \leq \frac{5}{6} + \frac{2\epsilon}{\sqrt{3\pi}}$, then $\psi_k \geq \epsilon$. If $0.a > \frac{5}{6} + \frac{2\epsilon}{\sqrt{3\pi}}$, then $\psi_k < \epsilon$. Proof of this is in appendix C.

Chapter 4

Quantum-Search for Violated Constraints

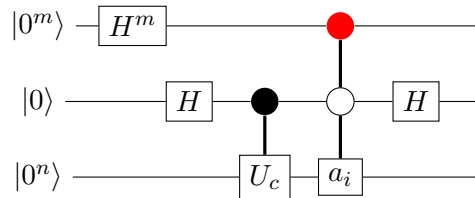
This section presents the main result of this research: a quantum-search circuit for violated constraints. Depicted here is the full circuit diagram of the algorithm and the rest of this section shows the effectiveness of this algorithm to find a violated constraint if such a constraint exists.



4.1 Interference Subcircuit

The subcircuit labeled $\mathcal{S}$ in the diagram above encodes the sign of $\langle a_i | c \rangle$ for a given c and the rows a_i in a state of super position. It does this by calling on a QRAM gate called a_i . This subcircuit is really several iterations of the interference subroutine described in §3.1.

The smallest part of our circuit is



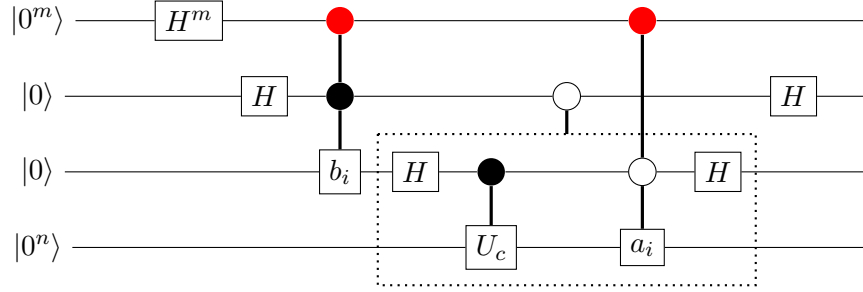
Here, the first register will enter a state of equal entanglement. This means it is in a state of superposition over all possible binary strings of length M . This superposition will be used to call the row a_i from our QRAM system. The second register will be the qubit which

encodes the inner-product of our center c and the row a_i . The final register is the workspace which is used to encode the quantum state of our center and a_i . The quantum state after running this sub circuit will be

$$|i\rangle \otimes \left(\frac{\sqrt{\frac{1}{2}(1 + \langle a_i|c\rangle)}}{\sqrt{\frac{1}{2}(1 - \langle a_i|c\rangle)}} \right) \otimes |\psi\rangle.$$

The state $|\psi\rangle$ is unimportant and does not need to be described in detail.

The next layer of the subcircuit will be



The gate b_i will encode the state $\left(\frac{\sqrt{\frac{1}{2}(1 + b_i)}}{\sqrt{\frac{1}{2}(1 - b_i)}} \right)$ when called. This means this circuit will generate the quantum state

$$\frac{1}{2}|0\rangle \otimes \left(\frac{\sqrt{\frac{1}{2}(1 + \langle a_i|c\rangle)} + \sqrt{\frac{1}{2}(1 + b_i)}}{\sqrt{\frac{1}{2}(1 - \langle a_i|c\rangle)} + \sqrt{\frac{1}{2}(1 - b_i)}} \right) + \frac{1}{2}|1\rangle \otimes \left(\frac{\sqrt{\frac{1}{2}(1 + \langle a_i|c\rangle)} - \sqrt{\frac{1}{2}(1 + b_i)}}{\sqrt{\frac{1}{2}(1 - \langle a_i|c\rangle)} - \sqrt{\frac{1}{2}(1 - b_i)}} \right).$$

Therefore, if we can identify the sign of the amplitude of state $|10\rangle$, then we will know the sign of $\langle a_i|c\rangle - b_i$. If the amplitude is negative, then we know constraint i has been violated.

4.2 No False Positives

We can test for negativity by using Nannicini's No False Positives test (NFP). This routine takes the unitary B , and index k , and an accuracy ϵ as input. For our purpose, the unitary will be defined by the circuit $\mathcal{A}$ and the index k will be the binary 10. Our value of epsilon will be defined by the ellipsoid method, requiring E digits of accuracy. The NFP routine will then need $p = \lceil \log_2(\frac{\sqrt{3} \cdot \pi}{\epsilon}) + 2 \rceil$ qubits in the first register for accuracy. For ellipsoid method, this means $\epsilon = 2^{-E}$ as defined in 2.3.1 of this work. Here, E is an integer.

This means we will use,

$$\begin{aligned} p &= \lceil \log_2(\frac{\sqrt{3} \cdot \pi}{\epsilon}) + 2 \rceil \\ &= \lceil \log_2(\sqrt{3} \cdot \pi) - \log_2(2^{-E}) + 2 \rceil \\ &= \lceil 2.444 + E + 2 \rceil \\ &= E + 5 \end{aligned}$$

This will return the bit string α_i . As per the requirements of the NFP routine, we will use control gates to output $|1\rangle$. If the first qubit is $|0\rangle$, meaning $0.\alpha_i < \frac{1}{2}$, then we activate

the circuit that tests if $0.\alpha_i < \frac{1}{6} - \frac{2^{-E+1}}{\sqrt{3\pi}}$. If the first qubit is $|1\rangle$, then we activate the circuit that tests if $0.\alpha_i > \frac{5}{6} + \frac{2^{-E+1}}{\sqrt{3\pi}}$. This will require $\mathcal{O}(E)$ gates since we must place a control gate on each qubit.

4.3 Complexity

The ellipsoid algorithm requires us to determine if a constraint has been violated by the current point c . If no such violated constraint exists, then the point c is feasible. Since there are M constraints each of dimension N and each multiplication operation is on the order of E , the complexity of checking each constraint is $\mathcal{O}(MNE)$.

The quantum search algorithm assumes encoding a quantum state represented by a vector of length N and entries of size E will take at most $\mathcal{O}(\text{poly}(\log_2(N))E)$ time. This is the same as saying $\tilde{\mathcal{O}}(E)$ time. To get the equal entanglement from the H^M and H^p gates requires $\log_2(M)$ and $\mathcal{O}(\log_2(p))$ gates which are both logarithmic to the input size of the problem. The $\mathcal{S}$ sub-circuit is dominated by the encoding of $|c\rangle$ and $|a_i\rangle$. This means it runs in $\tilde{\mathcal{O}}(E)$ time. Both S_{000} and S_{010} are constant operations. The QFT^{-1} gate runs with $\mathcal{O}(\log_2^2(N))$ operations which means it will be suppressed by $\tilde{\mathcal{O}}$ notation. The Grover operator requires $\mathcal{O}(\sqrt{M})$ applications with each iteration requiring $\mathcal{O}(\log_2(M))$ gates. We then need to check if the returned row constraint a_i is violated. This is done classically, which requires $\mathcal{O}(NE)$ operations to calculate $a_i^T c$ and check if $a_i^T c < b_i$.

Thus, the final complexity of the quantum search for violated constraints will be $\tilde{\mathcal{O}}(\sqrt{M}(\log_2(M) + N)E)$.

Chapter 5

Quantum-Search Ellipsoid Method

5.1 The Algorithm

The Quantum-Search Ellipsoid Method (QSEM) runs exactly as the classical ellipsoid method except that it implements the above search algorithm. However, it is unlikely that a generic LP will have its data as quantum preparable information. Therefore, this section provides a description of how to convert a classical LP into a quantum encodeable LP.

Let A and b be a constraint matrix and vector with M constraints and N variables. We will create the matrix A' of size $M' \times N'$ with constraint vector b' such that we can use A' and b' for quantum search. Let $M' := 2^{\lceil \log_2(M) \rceil}$ and $N' := 2^{\lceil \log_2(N) \rceil + 1}$. This way, we can have a circuit with registers of size $m' := \log_2(M')$ and $n' := \log_2(N')$ to run the circuit. For each row of A , append $N' - N$ zeros to generate each row of A' . Let the entries of b' be the same as b . Add $M' - M$ additional constraint rows to A' of the form $[0, \dots, 0, 1]$ and corresponding 0 entries to b' . Then, for each row of A' , find $\alpha_i = \mathbf{norm}(a'_i)$, normalize each row, and set $b'_i := b'_i / \alpha_i$. If no entry of b' has magnitude greater than 1, then proceed to the

encoding of c_0 . Otherwise, let $\beta = 2\mathbf{norm}(b')$ and set $b' := b' / \beta$ ensuring that $\begin{pmatrix} \sqrt{\frac{1}{2}(1 + b'_i)} \\ \sqrt{\frac{1}{2}(1 - b'_i)} \end{pmatrix}$ is an encodeable quantum state. Set $c_0 = (0, \dots, 0)^T \in \mathbb{R}^{M'}$ $c'_0 = |0^{M'-1}\rangle \otimes |1\rangle$. This means that if c'_0 violates row j of A' , then c_0 violates row j of A .

We will call the quantum search routine **QSearch** when used in the ellipsoid method. We will run **QSearch**(A', b', c'_0) to find the index of a violated row. Let j be the index produced by **QSearch**. If $a_j^T c_0 \geq b_j$, repeat **QSearch** until a violated constraint is found. Else, calculate c_1 and D_1 in the standard way. Then, set the first N entries of $c'_1 = c_1 / \beta$ and to some positive value such that c'_1 is a unit vector. All other entries should be left at 0. From there, repeat this process for each iteration until t_{max} is reached.

Algorithm 2 Quantum-Search Ellipsoid Method for Feasibility

- 1: **Input:** An $M \times N$ matrix A , an $M \times 1$ vector b , and a number U which is an upper bound on the magnitude of the entries of A and b
 - 2: **Output:** A vector x of length N such that $Ax \geq b$ or a statement that such a vector does not exist.
 - 3: **procedure** INITIALIZE
 - 4: $v := N^{-N}(NU)^{-N^2(N+1)}$; smallest possible volume of P
 - 5: $V := (2N)^N(NU)^{N^2}$; P 's vertices are in an origin-centered ball of volume V .
 - 6: $u := \lceil \log_2(U) \rceil$; encoding length of the entries of A and b .
 - 7: $r := \sqrt{N} \cdot 2^{(N^2+N)u-N^2}$
 - 8: $t_{\max} := \lceil 5N|\log_2(v)| + 5N^2|\log_2(2r)| \rceil$; maximum number of iterations.
 - 9: $c_0 := \mathbb{0}$; Center at origin
 - 10: $D_0 := r^2 I$
 - 11: $E_0 := (x_0, D_0)$
 - 12: $t := 0$
 - 13: **procedure** MAIN LOOP
 - 14: **if** $t \geq t_{\max}$, **then**
 - 15: *Output:* $P = \emptyset$
 - 16: Use **QSeach** to find a violated row a_i of A .
 - 17: **if** a_i does not exist **then**
 - 18: *Output:* $x := c_t$
 - 19: **if** $\exists a_i$ s.t. $a_i^T c_t < b_i$, **then**
 - 20: $c_{t+1} \approx c_t + \frac{1}{N+1} \cdot \frac{D_t a_i}{\sqrt{a_i^T D_t a_i}}$
 - 21: $D_{t+1} \approx \frac{2N^2+3}{2N^2} (D_t - \frac{2}{N+1} \cdot \frac{D_t a_i a_i^T D_t}{a_i^T D_t a_i})$
 - 22: $t := t + 1$
-

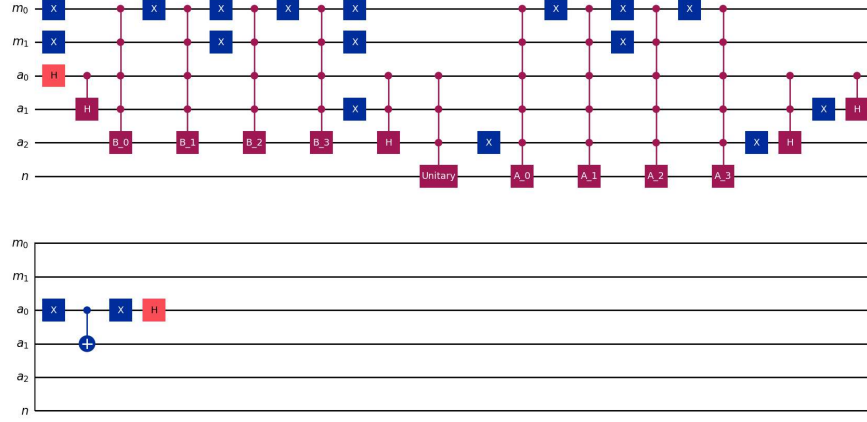


Figure 5.1: The subcircuit S with QRAM displayed

5.2 Implementation

The availability, and physical possibility, of robust quantum computers at this time limits the ability to run this algorithm on true quantum hardware. Thankfully, IBM has created the Qiskit package as a means to create and simulate quantum programs in Python [Han25]. These simulations were run with a 12th Gen Intel Core i9-12900KF 3.20 GHz CPU with 16 GB of RAM. Due to the computational power required to simulate quantum systems, the following demonstration is small compared to standard benchmarks.

5.2.1 Code Implementation

There are four parts of the circuit which need to be implemented: the S section, the Quantum Phase Estimation, the key generation, and Grover's iterations.

The QRAM needed to encode both A and b is not available as a built in function of qiskit. This is why the use of not-gates, represented by X , is so prevalent in the circuit. If genuine QRAM were available, the cost of encoding the relevant data would be far less than it was in this simulation. We can see the full circuit which encodes $\sqrt{\frac{1}{2}}(1 - \langle a_i | c \rangle) - \sqrt{\frac{1}{2}}(1 - \langle a_i | b_i \rangle)$ for each i using QRAM in figure 5.1

The most important part of phase estimation is the gate Q which is depicted here. These unitary gates are not generated by writing out their matrix representations. Rather, qiskit allows us to slot child quantum circuits into larger structures. This means in order to implement $-S$ as a gate, we instead use $(-I)S$. Thankfully, quantum phase estimation is a built in function for the qiskit package.

This is followed by a simple set of control gates which can provide the output for the NFP subroutine. The circuit before we apply the Grover iterations will look much like figure 5.3.

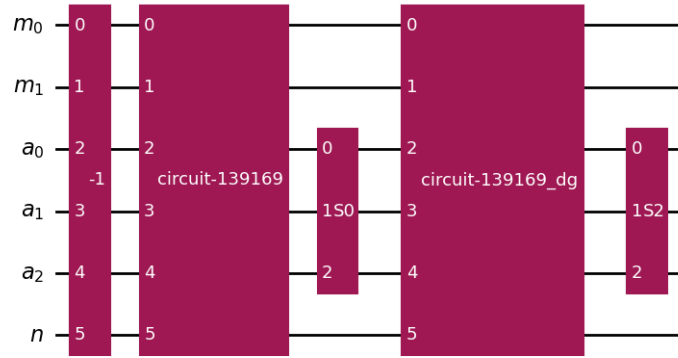


Figure 5.2: The Q subcircuit used in Amplitude Estimation

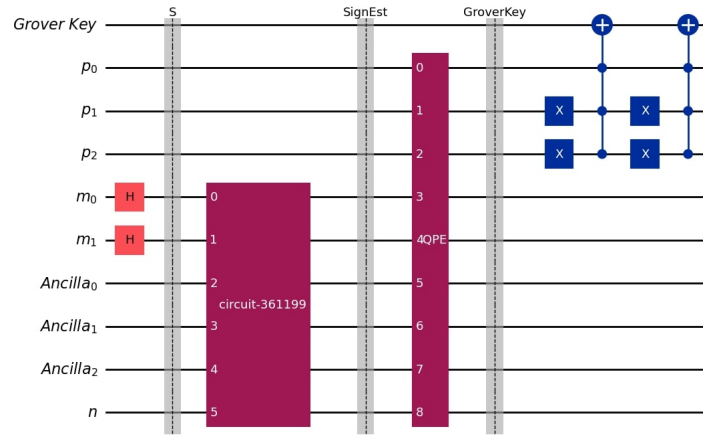


Figure 5.3: The NFP encoded with $C - Not$ Gates

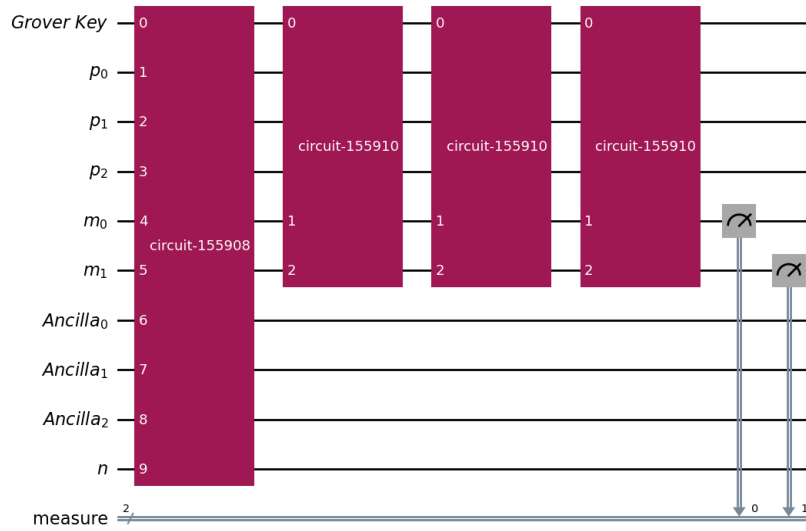


Figure 5.4: Final Circuit Output

The measurement gate in this graphic was used to observe the state of the additional qubit used as the key to activate the oracle in Grover's search. This is done by running several trials over the circuit. It reveals the ability of the circuit before the Grover iterations to be used for quantum counting, which can reduce the number of iterations needed.

The final circuit will look something like what is shown in figure 5.4.

5.2.2 Computational Results

Toy Search

This code was used to find the index of the violated constraint for the inputs

$$A = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, b = \begin{pmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{pmatrix}, c = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.$$

This is a very small instance with very simple data, which is why it is labeled as a toy search. Over 10 trials, it took an average of 2.8 applications of the Grover iterative in order to find the violated constraint.

Easy LP

This is an LP that is easy to solve since the feasible region boxes in the center of the ellipse. This means that so long as the center of the ellipse moves, it will find the feasible region. We also don't need to rescale the constraint data since it is already quantum encodeable.

$$A = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} & 0 & 0 \\ \frac{-1}{\sqrt{2}} & \frac{-1}{\sqrt{2}} & 0 & 0 \end{pmatrix}, b = \begin{pmatrix} 0 \\ 0 \\ -1 \\ -1 \\ \frac{\sqrt{2}}{4} \\ \frac{-1}{\sqrt{2}} \end{pmatrix}$$

The careful reader will see that some of these constraints are redundant. However, this is a scenario where the benefits of using quantum search shines since it make the rotation performed by Grover's search a smaller number. This means as we approach the optimal number of Grover iterates, the odds our search superposition will collapse to a good state will be even closer to 1.

Having reached the limits of this projects computational power, the simulated algorithm only able to run a single trial of the ellipsoid method to solve this problem. However, it did so over two iterations with only one application of the Grover iterative each time.

5.2.3 Practicality

While there is theoretical substance to the use of quantum computation for the ellipsoid method, current hardware limits any true implementation. The number of qubits will be dominated by the factor $p \sim \mathcal{O}(N^3 \ln U)$. If the number of variables is $N = 10$, then one has already reached the limit of the largest quantum computers currently operational.

If one were to run more intensive tests on this subroutine, then the set of benchmarks taken from [Amm+23] would be a reasonable comparison within the field. This work showed the impracticable nature of the quantum simplex method developed by Nannicini. This is due to the fact the operational speed of quantum gates on logical qubits is not ye fast enough to compete with classical computers.

Quantum hardware will need to improve in scale, speed, and stability by severally orders of magnitude before these algorithms become anything more than theoretically interesting.

Given the advancement of the field in the past decade, this is development of hardware is not an unreasonable expectation.

Chapter 6

Conclusion

6.1 Results

Thus, here presented is a quantum search subroutine which can speed up the ellipsoid method to detect feasibility in an LP with both theoretical justification and experimental data. It provides a quadratic speed-up in the search subroutine, taking the cost of finding a violated row from $\mathcal{O}(MNE)$ to $\tilde{\mathcal{O}}(\sqrt{M}(\log_2(M) + N)E)$ consistent with the speed-ups normally found in the field of quantum computation.

In terms of practicality, this subroutine is highly dependent upon coherent and fault-tolerant logical qubits. At the time of writing, such hardware does not exist. The best current quantum computers only have a few hundred qubits and any real world problem would require thousands if not tens of thousands. However, Microsoft [Aas+25] and IBM [Yod+25] claim that such devices will be available within the next decade.

6.2 Future Research

The next obvious steps in research are to determine a which can effectively calculate the ellipses used in EM. This would require the ability to calculate both

$$x_{t+1} := x_t + \frac{1}{n+1} \cdot \frac{D_t a_i}{\sqrt{a_i^T D_t a_i}}; D_{t+1} := \frac{n^2}{n^2 - 1} \left(D_t - \frac{2}{n+1} \cdot \frac{D_t a_i a_i^T D_t}{a_i^T D_t a_i} \right)$$

efficiently on quantum hardware. This step is promising since $\frac{D_t a_i}{\sqrt{a_i^T D_t a_i}}$ bears striking resemblance to the way one calculates projective measurement especially since we know D_t to be a non-negative definite matrix. Direct calculation and recovery of data bears no benefit compared to classical subroutines for this calculation. However, a well designed algorithm should be able to avoid this issue.

Instead of a single violated a_i being used in the above calculations, one could try to implement a superposition of all violated a_i . That way, instead of the sequence of ellipses moving in jagged directions, all of which are perpendicular to individual constraints, the sequence moves in a more direct line to the feasible region. This may cut down on the total number of iterations required by the ellipsoid method which could make it more competitive with more popular methods.

One long term goal within the field of quantum LP is to build a method which is designed specifically for a quantum computer, as opposed to adapting classical methods with quantum subroutines. Such a method will certainly receive more research as time goes on since there is potential for exponential speed up in optimization problems. It would also need to out perform the quantum QUBO solvers which are implemented today on available hardware.

Bibliography

- [Aas+25] David Aasen et al. *Roadmap to fault tolerant quantum computation using topological qubit arrays*. 2025. arXiv: 2502.12252 [quant-ph]. URL: <https://arxiv.org/abs/2502.12252>.
- [Amm+23] Sabrina Ammann et al. *Realistic Runtime Analysis for Quantum Simplex Computation*. 2023. arXiv: 2311.09995 [quant-ph].
- [Aug+23] Brandon Augustino, Giacomo Nannicini, Tamás Terlaky, and Luis F. Zuluaga. “Quantum Interior Point Methods for Semidefinite Optimization”. In: *Quantum* 7 (Sept. 2023), p. 1110. ISSN: 2521-327X. DOI: 10.22331/q-2023-09-11-1110. URL: <http://dx.doi.org/10.22331/q-2023-09-11-1110>.
- [Ben82] Paul Benioff. “Quantum mechanical hamiltonian models of turing machines”. In: *Journal of Statistical Physics* 29.3 (Nov. 1982), pp. 515–546. ISSN: 1572-9613. DOI: 10.1007/BF01342185. URL: <https://doi.org/10.1007/BF01342185>.
- [Bra+02] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp. *Quantum amplitude amplification and estimation*. 2002. URL: <http://dx.doi.org/10.1090/conm/305/05215>.
- [BT97] D. Bertsimas and J.N. Tsitsiklis. *Introduction to linear optimization*. Athena Scientific, 1997.
- [CGK98] Isaac L. Chuang, Neil Gershenfeld, and Mark Kubinec. “Experimental Implementation of Fast Quantum Searching”. In: *Phys. Rev. Lett.* 80 (15 Apr. 1998), pp. 3408–3411. DOI: 10.1103/PhysRevLett.80.3408. URL: <https://link.aps.org/doi/10.1103/PhysRevLett.80.3408>.
- [CM20] P A M Casares and M A Martin-Delgado. “A quantum interior-point predictor–corrector algorithm for linear programming”. In: *Journal of Physics A: Mathematical and Theoretical* 53.44 (Oct. 2020), p. 445305. ISSN: 1751-8121. URL: <http://dx.doi.org/10.1088/1751-8121/abb439>.
- [Dan82] George B. Dantzig. “Reminiscences about the origins of linear programming”. In: *Operations Research Letters* 1.2 (1982), pp. 43–48. ISSN: 0167-6377. DOI: [https://doi.org/10.1016/0167-6377\(82\)90043-8](https://doi.org/10.1016/0167-6377(82)90043-8). URL: <https://www.sciencedirect.com/science/article/pii/0167637782900438>.
- [Fey82] Richard P. Feynman. “Simulating physics with computers”. In: *International Journal of Theoretical Physics* 21.6 (June 1982), pp. 467–488. ISSN: 1572-9575. DOI: 10.1007/BF02650179. URL: <https://doi.org/10.1007/BF02650179>.
- [GLS88] Martin Grötschel, László Lovász, and Alexander Schrijver. *Geometric Algorithms and Combinatorial Optimization*. Springer-Verlag Berlin Heidelberg, 1988.

- [Gro96] Lov K. Grover. *A fast quantum mechanical algorithm for database search*. 1996. arXiv: quant-ph/9605043 [quant-ph]. URL: <https://arxiv.org/abs/quant-ph/9605043>.
- [Han25] Benjamin Hanzsek-Brill. *Quantum Optimization*. <https://github.com/Ben-RHBrill/Quantum-Optimization>. 2025.
- [JAJ21] Adeline Jordon, Prashanti Priya Angara, and Saasha Joshi. “Implementing the Simplex Method with Grover’s Search”. In: *2021 IEEE International Conference on Quantum Computing and Engineering (QCE)*. 2021, pp. 435–436. DOI: 10.1109/QCE52317.2021.00066.
- [Joh+11] M. W. Johnson et al. “Quantum annealing with manufactured spins”. In: *Nature* 473.7346 (May 2011), pp. 194–198. ISSN: 1476-4687. DOI: 10.1038/nature10012. URL: <https://doi.org/10.1038/nature10012>.
- [Kan60] L. V. Kantorovich. “Mathematical Methods of Organizing and Planning Production”. In: *Management Science* 6.4 (1960), pp. 366–422. ISSN: 00251909, 15265501. URL: <http://www.jstor.org/stable/2627082> (visited on 08/03/2025).
- [Kha80] Leonid Khachiyan. “A Polynomial-Time Algorithm for Solving Linear Programs”. In: *Math. Oper. Res.* 5.1 (Feb. 1980), p. iv. ISSN: 0364-765X. DOI: 10.1287/moor.5.1.iv. URL: <https://doi.org/10.1287/moor.5.1.iv>.
- [KP18] Iordanis Kerenidis and Anupam Prakash. *A Quantum Interior Point Method for LPs and SDPs*. 2018. arXiv: 1808.09266 [quant-ph].
- [Man80] I?U?ri Ivanovich Manin. *Vychislimoe I Nevychislimoe (Computable and Non-Computable)*. Moscow: Soviet Radio, 1980.
- [Nan20] Giacomo Nannicini. *An Introduction to Quantum Computing, Without the Physics*. 2020. arXiv: 1708.03684 [cs.DM]. URL: <https://arxiv.org/abs/1708.03684>.
- [Nan22] Giacomo Nannicini. *Fast quantum subroutines for the simplex method*. 2022. arXiv: 1910.10649 [quant-ph].
- [NC00] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2000.
- [Sho97] Peter W. Shor. “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer”. In: *SIAM Journal on Computing* 26.5 (Oct. 1997), pp. 1484–1509. ISSN: 1095-7111. DOI: 10.1137/S0097539795293172. URL: <http://dx.doi.org/10.1137/S0097539795293172>.
- [Str87] Gilbert Strang. “Karmarkar’s algorithm and its place in applied mathematics”. In: *The Mathematical Intelligencer* (1987).
- [SZ15] Gerard Sierksma and Yori Zwols. *Linear and Integer Programming: Theory and Practice*. CRC Press, 2015.
- [Yod+25] Theodore J. Yoder et al. *Tour de gross: A modular quantum computer based on bivariate bicycle codes*. 2025. arXiv: 2506.03094 [quant-ph]. URL: <https://arxiv.org/abs/2506.03094>.
- [Zha+19] Liming Zhao, Zhikuan Zhao, Patrick Rebentrost, and Joseph Fitzsimons. *Compiling basic linear algebra subroutines for quantum computers*. 2019. arXiv: 1902.10394 [quant-ph]. URL: <https://arxiv.org/abs/1902.10394>.

Appendix A

Grover's Algorithm

This is adapted from the text by Nielsen and Chuang [NC00, §6.1].

Algorithm 3 Grover's Algorithm

- 1: **Input:** A boolean function f on m bits with μ solutions and the oracle gate U_f such that $U_f|x\rangle = (-1)^{f(x)}|x\rangle$
 - 2: **Output:** A binary string y of length m such that $f(y) = 1$
 - 3: **procedure**
 - 4: Create two quantum registers. The first of m and the second on n qubits where n is the workspace needed for U_f .
 - 5: Let $G := H^m \otimes (2|0\rangle\langle 0| - I) \otimes H^m \otimes U_f$
 - 6: Apply H^m to the first register.
 - 7: Apply G for $\lceil \frac{\pi}{4} \sqrt{\frac{m}{\mu}} \rceil$ iterations.
 - 8: Measure the first register and return the outcome y .
-

Take $|\psi\rangle = H^m|0^m\rangle$. We may rewrite $|\psi\rangle$ as a combination $|\psi\rangle$ of states where $f(\psi_0) = 0$ and $|\psi_1\rangle$ being the converse. Therefore, one can observe $|\psi\rangle = \sqrt{\frac{m-\mu}{m}}|\psi_0\rangle + \sqrt{\frac{\mu}{m}}|\psi_1\rangle$. In fact, we can imagine $|\psi\rangle$ existing as a vector on the $2D$ plane where $|\psi_0\rangle$ and $|\psi_1\rangle$ are the basis vectors. This means $|\psi\rangle = \cos(\theta)|\psi_0\rangle + \sin(\theta)|\psi_1\rangle$

Application of U_f represents a reflection over $|\psi_0\rangle$ and $H^m \otimes (2|0\rangle\langle 0| - I) \otimes H^m = 2|\psi\rangle\langle\psi| - I$ is a reflection over the equally entangled state $|\psi\rangle$. Therefore,

$$G|\psi\rangle = \cos(3\theta)|\psi_0\rangle + \sin(3\theta)|\psi_1\rangle, G^k|\psi\rangle = \cos((2k+1)\theta)|\psi_0\rangle + \sin((2k+1)\theta)|\psi_1\rangle.$$

If $k = \frac{\pi}{4} \sqrt{\frac{m}{\mu}}$, then $\cos((2k+1)\theta) = 1$. Let $k = \lceil \frac{\pi}{4} \sqrt{\frac{m}{\mu}} \rceil$ be the number of applications of G to the circuit which maximizes the probability of a good result. Since the oracle is a black box operation in this case, any complexity calculation in practice will likely depend quite heavily on the complexity of the oracle. Thus, we get a complexity of $\mathcal{O}(\sqrt{\frac{m}{\mu}})$ which is quadratically faster than the classical method of checking each entry.

If the number μ is unknown, it has been shown that slowly but exponentially increasing the number of Grover iterations will still give a complexity dependent on $\sqrt{\frac{m}{\mu}}$ [Bra+02].

Appendix B

Amplitude Estimation

This is adapted from Brassard's work on amplitude estimation and amplification [Bra+02].

Suppose we have some boolean function $f : \{0, 1\}^p \rightarrow \{0, 1\}$. We call a string $j \in \{0, 1\}^p$ a *good* string if $f(j) = 1$. Given a quantum state $|\psi\rangle$, we would like to determine the a which is the probability $|\psi\rangle$ collapses to a good state upon measurement. We can write $|\psi\rangle = |\psi_1\rangle + |\psi_0\rangle$ representing the sum of good and bad states which compose $|\psi\rangle$. Therefore, $a = \langle \psi_1 | \psi_1 \rangle$. If instead of writing $|\psi\rangle$ in the computational basis we chose to express it in the eigenvalue basis of $\mathcal{A}$, we can write $|\psi\rangle = \frac{-i}{\sqrt{2}}(e^{i\theta_a}|\psi_+\rangle - e^{-i\theta_a}|\psi_-\rangle)$ where $|\psi_+\rangle$ and $|\psi_-\rangle$ are the good and bad superpositions expressed in the new basis. It should also be noted that $\sin^2(\theta_a) = a$.

In order for this algorithm to work, we assume we have access to a gate $\mathcal{A}$ such that $\mathcal{A}|0\rangle = |\psi\rangle$. We also assume we have access to gates S_0 and S_f such that,

$$S_0|j\rangle = \begin{cases} -|j\rangle; j = 0 \\ |j\rangle; j \neq 0 \end{cases}, S_f|j\rangle = \begin{cases} -|i\rangle; f(j) = 1 \\ |i\rangle; f(j) = 0 \end{cases}$$

We define $Q = -\mathcal{A}S_0\mathcal{A}^{-1}S_f$. This is similar to the operator used for Grover's search algorithm. The gate $\Lambda_p(Q)(|k\rangle|\phi\rangle) = |k\rangle Q^k|\phi\rangle$ where k is a binary number of p bits and $|\phi\rangle$ is a state on n bits.

We note that $Q^k|\psi\rangle = e^{i\theta_a}e^{2ik\theta_a}|\psi_+\rangle - e^{-i\theta_a}e^{-2ik\theta_a}|\psi_-\rangle$

We will show the effectiveness of this algorithm by following the unitary transform defined by the circuit on the initial state. We observe,

$$\begin{aligned}
& (\mathbf{QFT}^{-1} \otimes I^n)(\Lambda_p(Q))(H^p \otimes I^n)(I^p \otimes \mathcal{A})|0^p\rangle|0^n\rangle \\
&= (\mathbf{QFT}^{-1} \otimes I^n)(\Lambda_p(Q))(H^p \otimes I^n)|0^p\rangle|\psi\rangle \\
&= (\mathbf{QFT}^{-1} \otimes I^n)(\Lambda_p(Q))\frac{1}{\sqrt{p}}\sum_{j=0}^{p-1}|j\rangle|\psi\rangle \\
&= (\mathbf{QFT}^{-1} \otimes I^n)\frac{1}{\sqrt{p}}\sum_{j=0}^{p-1}|j\rangle Q^j|\psi\rangle \\
&= (\mathbf{QFT}^{-1} \otimes I^n)\frac{1}{\sqrt{p}}\sum_{j=0}^{p-1}|j\rangle(e^{i\theta_a}e^{2ij\theta_a}|\psi_+\rangle - e^{-i\theta_a}e^{-2ij\theta_a}|\psi_-\rangle) \\
&= |\theta_a\rangle|\phi\rangle
\end{aligned}$$

Here, $|\phi\rangle$ is a quantum state which does not contain relevant information. We measure the first register which returns the binary θ_a .

Algorithm 4 Amplitude Estimation

- 1: **Input:** A unitary gate $\mathcal{A}$ acting on N qubits, boolean function f , precision p .
 - 2: **Output:** The p bit approximation $\tilde{a}$ of the probability a that $|\psi\rangle = \mathcal{A}|0^N\rangle$ will collapse to a binary string j such that $f(j) = 1$
 - 3: **procedure** INITIALIZE
 - 4: Create two quantum registers. The first of p qubits and the second of n qubits.
 - 5: Create the gates S_0 and S_f both acting on n qubits.
 - 6: Create gate $Q = -\mathcal{A}S_0\mathcal{A}^{-1}S_f$
 - 7: Set the state of the system to $|0^p\rangle|0^n\rangle$
 - 8: **procedure** MAIN
 - 9: Apply $\mathcal{A}$ to the second register.
 - 10: Apply H^p to the first register.
 - 11: Apply $\Lambda_p(Q)$ to the state with the first register acting as the control.
 - 12: Apply $\mathbf{QFT}^{-1}$ to the first register.
 - 13: Measure the first register returning binary number θ_a
 - 14: Output $\tilde{a} = \mathbf{sin}^2(\theta_a)$
-

Appendix C

Sign Estimation

Adapted from the work of Nannicini on quantum simplex method [Nan22].

Algorithm 5 Sign Estimation NFP

- 1: **Input:** Some unitary U_ϕ acting on n qubits such that $U|0^n\rangle = |\phi\rangle$ where the amplitudes of $|\phi\rangle$ are real, some index $k \in \{0, \dots, 2^n - 1\}$, and precision ϵ .
 - 2: **Output:** 0 if $\phi_k \geq -\epsilon$ with probability at least $\frac{3}{4}$
 - 3: Let V be a gate such that $V|0^n\rangle = |k\rangle$.
 - 4: Compute $|\psi\rangle$ to be state produce by running the interference of quantum states routine from 3.1 on U and V .
 - 5: Apply Brassard's Amplitude Estimation to the state $|0\rangle \otimes |k\rangle$ with $\lceil \log(\frac{9\sqrt{3}\pi}{\epsilon}) \rceil + 2$ qubits of accuracy. Let a be the bit string produced the phase estimation portion of the algorithm.
 - 6: If $0.a \leq \frac{1}{6} - \frac{2\epsilon}{3\sqrt{3}\pi}$, return 1. If $0.a \geq \frac{1}{2}$ and $1 - 0.a > \frac{1}{6} - \frac{2\epsilon}{3\sqrt{3}\pi}$, return 1. Else, return 0.
-

The interference routine of step 4 will produce $\sqrt{\frac{1}{2}(1 + \phi_k)}$ and we use amplitude estimation to determine its magnitude. Suppose $\phi_k \leq -\epsilon$. This means for θ approximated by Amplitude Estimation $\sin(\pi\theta) \leq \frac{1}{2}(1 - \epsilon)$. Therefore, $\theta \leq \frac{1}{6} - \frac{7\epsilon}{9\sqrt{3}\pi}$. If $\tilde{\theta}$ is the actual string produced by Brassard's algorithm, then $|\theta - \tilde{\theta}| \leq \frac{\epsilon}{9\sqrt{3}\pi}$ with probability at least $\frac{3}{4}$ [NC00, §5.2]. This means $\tilde{\theta} \leq \frac{1}{6} - \frac{2\epsilon}{3\sqrt{3}\pi}$. The rest of the proof is similar.